

Métricas de Código-fonte

Principais ideias

- ↴ Quantificar propriedades de um projeto de software
- ↴ Necessitam do código-fonte implementado
- ↴ Permitem avaliar a qualidade do código
- ↴ Não são muito usadas na prática:
 - ↵ Avaliação das métricas são subjetivas
 - ↵ Faixas aceitáveis para as métricas podem variar dependendo do tipo/domínio do sistema

Vamos estudar

- ↵ Tamanho
- ↵ Coesão
- ↵ Acoplamento
- ↵ Complexidade

Tamanho

- ↴ Medida mais popular: linhas de código (LOC)
- ↴ Importante especificar se linhas em branco e comentários são contados
- ↴ Não deve ser usada para medir produtividade de programadores:

“Um dos dias mais produtivos da minha vida foi quando eu deletei 1.000 linhas de código do sistema”

Ken Thompson (desenvolvedor do Unix)

Coesão

- ↴ LCOM (*Lack of Cohesion Between Methods*) - falta de coesão
- ↴ Quanto mais alta, pior é a classe
- ↴ $M(C) = \{ (f1, f2) \mid f1 \text{ e } f2 \text{ são métodos da classe } C \}$
- ↴ $A(f) = \text{conjunto de atributos acessados por um método } f$
- ↴ $LCOM = | \{ (f1, f2) \text{ in } M(C) \mid A(f1) \text{ e } A(f2) \text{ são conjuntos disjuntos} \} |$

LCOM - exemplo

```
class A {  
  
    int a1;  
    int a2;  
    int a3;  
  
    void m1() {  
        a1 = 10;  
        a2 = 20;  
    }  
  
    void m2() {  
        System.out.println(a1);  
        a3 = 30;  
    }  
  
    void m3() {  
        System.out.println(a3);  
    }  
}
```

Pares de Métodos	Conjuntos A	Interseção dos Conjuntos A
(m1,m2)	A(m1) = {a1,a2} A(m2) = {a1,a3}	{a1}
(m1,m3)	A(m1) = {a1,a2} A(m3) = {a3}	{ }
(m2,m3)	A(m2) = {a1,a3} A(m3) = {a3}	{a3}

Logo, $LCOM(C) = 1$

LCOM parte do pressuposto que, em uma classe coesa, qualquer par de métodos deve acessar pelo menos um atributo em comum.

LCOM - considerações

↴ Desconsiderar no cálculo:

↵ Construtores

↵ Getters e Setters

↴ Métricas propostas por Chidamber e Kemerer (1991) (CK) - há outros autores com algumas variações nestes cálculos

Acoplamento

- ↴ CBO (*Coupling Between Objects*) - também proposta por CK
- ↴ Dada uma classe A, CBO conta o número de classes das quais A depende de forma sintática (ou estrutural)
- ↴ Diz-se que A depende de uma classe B quando:
 - ↵ A chama um método de B
 - ↵ A acessa um atributo público de B
 - ↵ A herda de B
 - ↵ A declara uma variável local, um parâmetro ou um tipo de retorno do tipo B
 - ↵ A captura uma exceção do tipo B
 - ↵ A levanta uma exceção do tipo B
 - ↵ A cria um objeto do tipo B.

CBO - Exemplo

```
class A extends T1 implements T2 {  
  
    T3 a;  
  
    T4 metodo1(T5 p) throws T6 {  
        T7 v;  
        ...  
    }  
  
    void metodo2() {  
        T8 = new T8();  
        try {  
            ...  
        }  
        catch (T9 e) { ... }  
    }  
}
```

$$\text{CBO}(A) = 9$$

Observação: CBO não distingue as classes das quais uma classe depende. Por exemplo, tanto faz se a dependência é para uma classe da biblioteca de Java ou uma classe da própria aplicação que está sendo desenvolvida

Complexidade

- ↴ Complexidade de uma classe é dada pela soma da complexidade dos seus métodos
- ↴ Complexidade dos métodos é calculada pela complexidade ciclomática
 - ↵ É dada pelo “número de comandos de decisão em um método” + 1
 - ↵ Vamos estudar com mais detalhes quando falarmos de Testes

Outras métricas

- ↴ Profundidade da Árvore de Herança
- ↴ Número de Classes Ancestrais e/ou interfaces
- ↴ Respostas de uma classe (RFC): # de métodos que podem ser executados a partir de métodos da classe
- ↴ Sugestão de CK, investigar a classe toda vez que ocorrer pelo menos 2 dos seguintes critérios:
 - ↵ RFC > 100 ou RFC maior que 5 vezes o número de métodos da classe
 - ↵ CBO > 5
 - ↵ Complexidade > 100

Referências

- ↴ PRESSMAN, R. S.. Engenharia de Software. Makron Books. 1995.
- ↴ Valente, M. T. Engenharia de Software Moderna. Editora Independente, 2020 (<https://engsoftmoderna.info/>)